

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)

IDE: Google Colab

Theory:

To preprocess your text simply means to bring your text into a form that is predictable and analyzable for your task. A task here is a combination of approach and domain. Machine Learning needs data in the numeric form. We basically used encoding technique (Bag Of Word, Bi-gram, n-gram, TF-IDF, Word2Vec) to encode text into numeric vector. But before encoding we first need to clean the text data and this process to prepare (or clean) text data before encoding is called text preprocessing, this is the very first step to solve the NLP problems.

Tokenization:

Tokenization is about splitting strings of text into smaller pieces, or "tokens". Paragraphs can be tokenized into sentences and sentences can be tokenized into words.

Filtration:

Similarly, if we are doing simple word counts, or trying to visualize our text with a word cloud, stopwords are some of the most frequently occurring words but don't really tell us anything. We're often better off tossing the stopwords out of the text. By checking the Filter Stopwords option in the Text Pre-processing tool, you can automatically filter these words out.

Stemming:

Stemming is the process of reducing inflection in words (e.g. troubled, troubles) to their root form (e.g. trouble). The "root" in this case may not be a real root word, but just a canonical form of the original word.

Stemming uses a crude heuristic process that chops off the ends of words in the hope of correctly actually be converted to trouble instead of trouble because the ends were just chopped off (ughh, how crude!). There are different algorithms for stemming. The most common algorithm, which is also known to be empirically effective for English, is Porter's Algorithm. Here is an example of stemming in action with Porter Stemmer:

	original word	stemmed words
0	connect	connect
'1	connected	connect
2	connection	connect
3	connections	connect
4	connects	connect

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

Stopword Removal

Stop words are a set of commonly used words in a language. Examples of stop words in English are "a", "the", "is", "are" and etc. The intuition behind using stop words is that, by removing low information words from text, we can focus on the important words instead.

For example, in the context of a search system, if your search query is 'what is text preprocessing?', you want the search system to focus on surfacing documents that talk about text preprocessing over documents that talk about what is. This can be done by preventing all words from your stop word list from being analyzed. Stop words are commonly applied in search systems, text classification applications, topic modeling, topic extraction and others. Stop word removal, while effective in search and topic extraction systems, showed to be non-critical in classification systems. However, it does help reduce the number of features in consideration which helps keep your models decently sized

Program (Code):

To be attached with

1. Tokenization

```
# @title
text=""" This is Narendra, currently i am listening the ai subject lab in the ma112 lab in the marwadi university where it
was located in the rajkot"""
# @title
#splitting in the words
text.split()
# @title
text[5]
# @title
import spacy #nlp library (gensim,nltk,spacy)
nlp=spacy.load('en_core_web_sm')
# @title
doc=nlp(text)
# @title
doc[5]
# @title
text="""I am Nishith Kotak, teaching the subject AI
to Semester 6 students at Marwadi University, India"""
doc=nlp(text)
#NER: Named Entity Recognition
for token in doc.ents:
    print(token.text, token.label_)
# @title
#POS Tags: Part of speech taggings
#text="Google is a very good search engine"
text="Let's google it"
doc=nlp(text)
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

```
for token in doc:
    print(token.text, token.pos_)
# @title
#Lemmatization vs stemming
text=""I studied the subject AI and then
meeting Mr. Sachin tomorrow in the meeting""
```

```
import nltk
from nltk.stem.porter import * ## stemming
p_stem=PorterStemmer()
for word in text.split():
    print(word,"-->", p_stem.stem(word))
# @title
#lemmatization
for token in nlp(text):
    print(token.text,"- ",token.pos_,
          "- ",token.lemma_)
# @title
#punctuation marks
import string
string.punctuation
# @title
#stopwords
from nltk.corpus import stopwords
nltk.download('stopwords')
print(set(stopwords.words('english')))
```

Results:

To be attached with

```
{'isn't', 'an', 'down', 's', 'now', 'haven't', 'same', 'you've', 'weren't', 'he'd', 'if', 'they', 'she'd', 'into', 'further', 'while',
'i'm', 'shan't', 'am', 'against', 'only', 'very', 'were', 'haven', 'through', 'couldn', 'he's', 'mustn', 'these', 'me', 'they'll',
'some', 'than', 'the', 'because', 'here', 'where', 'having', 've', 'been', 'we'd', 'that'll', 'any', 'has', 'she', 'what', 'from',
'hadn't', 'wasn', 'whom', 'won', 'again', 'should've', 'didn', 'until', 'just', 'so', 'of', 'below', 'you', 'we'll', 'yourself',
'myself', 'yourselves', 't', 'hasn't', 'was', 'its', 'being', 'few', 'with', 'no', 'during', 'ma', 'we're', 'such', 'both',
'between', 'aren't', 'by', 'mustn't', 'out', 'we', 'ours', 'he'll', 'won't', 'isn', 'it's', 'm', 'for', 'after', 'mightn't', 'you'll',
'is', 'hers', 'll', 'your', 'off', 'under', 'my', 'hasn', 'more', 'we've', 'theirs', 'do', 'how', 'above', 'those', 'and', 'as', 'ain',
'ourselves', 'this', 'other', 'our', 'y', 'o', 'can', 'i'd', 'needn't', 'her', 'themselves', 'him', 'i'll', 'needn', 'she's',
'shouldn', 'that', 'to', 'i've', 'mightn', 'most', 'itself', 'they've', 'had', 'are', 'doesn't', 'don't', 'doing', 'hadn', 'he',
'yours', 'it'll', 'himself', 'or', 'will', 'there', 'his', 'over', 'it', 'on', 'i', 'couldn't', 'who', 'weren', 'not', 'them', 'be',
'wasn't', 'did', 'too', 'you're', 'each', 'their', 'a', 'which', 'don', 'wouldn't', 'it'd', 'at', 'own', 'should', 'aren', 'in',
'about', 'does', 'shouldn't', 'nor', 'all', 'wouldn', 'why', 'she'll', 'have', 'd', 'up', 'when', 'didn't', 'then', 'but', 'herself',
'doesn', 'you'd', 'they're', 'they'd', 'shan', 'before', 'once', 're'}
```

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Unzipping corpora/stopwords.zip.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

Observation:

In this experiment, different NLP operations were performed using Python libraries like spaCy and NLTK. Initially, the text was split into words using the split() function. It was observed that this method is simple but does not handle punctuation or sentence structure properly.

Using spaCy, the text was converted into a document object, which allowed advanced processing. Named Entity Recognition (NER) was applied, and entities such as person names and locations were correctly identified. This shows that spaCy can extract meaningful information from text.

Part-of-Speech (POS) tagging was also performed, where each word was assigned a grammatical role like noun, verb, etc. The results were accurate even for informal sentences, showing the effectiveness of spaCy.

Further, stemming and lemmatization were compared. Stemming reduces words to their root form but sometimes produces incorrect or incomplete words. On the other hand, lemmatization gives meaningful base forms, making it more reliable.

Finally, punctuation and stopwords were identified using Python libraries. Stopwords are common words that do not add much meaning and are usually removed during preprocessing.

Post Lab Exercise:

1. Take your own document of 10 sentences and perform the same tasks. Attach code and screenshot of your output.
1. I am Narendra studying Artificial Intelligence at Marwadi University.
2. The university is located in Rajkot, Gujarat, India.
3. I enjoy learning about machine learning and deep learning.
4. My professor teaches AI in a very interactive way.
5. We perform experiments in the MA112 lab.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

6. Python is my favorite programming language.
7. I have completed projects on virtual mouse and smart parking system.
8. NLP is one of the most interesting topics for me.
9. I use libraries like spaCy and NLTK for text processing.
10. I want to become a skilled AI engineer in the future.

Code:-

@title Full NLP Experiment Code

Custom Text (10 sentences)

```
text = """I am pavan studying Artificial Intelligence at Marwadi University.
The university is located in Rajkot, Gujarat, India.
I enjoy learning about machine learning and deep learning.
My professor teaches AI in a very interactive way.
We perform experiments in the MA112 lab.
Python is my favorite programming language.
I have completed projects on virtual mouse and smart parking system.
NLP is one of the most interesting topics for me.
I use libraries like spaCy and NLTK for text processing.
I want to become a skilled AI engineer in the future."""
```

1. Basic Tokenization

```
print(" Tokens:")
print(text.split())
```

Access character

```
print("\n Character at index 10:")
print(text[10])
```

2. Load spaCy

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

```

import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp(text)

print("\n spaCy Token at index 10:")
print(doc[10])

# -----
# 3. Named Entity Recognition (NER)
print("\n Named Entities:")
for token in doc.ents:
    print(token.text, "-", token.label_)

# -----
# 4. POS Tagging
print("\n POS Tagging:")
for token in doc:
    print(token.text, "-", token.pos_)

# -----
# 5. Stemming
import nltk
from nltk.stem.porter import PorterStemmer

print("\n Stemming:")
p_stem = PorterStemmer()
for word in text.split():
    print(word, "-->", p_stem.stem(word))

# -----
# 6. Lemmatization

```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

```
print("\n Lemmatization:")
for token in doc:
    print(token.text, "-", token.pos_, "-", token.lemma_)
```

```
# -----
```

```
# 7. Punctuation
```

```
import string
print("\n Punctuation:")
print(string.punctuation)
```

```
# -----
```

```
# 8. Stopwords
```

```
from nltk.corpus import stopwords
nltk.download('stopwords')
```

```
print("\n Stopwords:")
print(set(stopwords.words('english')))
```

Output:-

Tokens:-

```
['I', 'am', 'pavan', 'studying', 'Artificial', 'Intelligence', 'at', 'Marwadi', 'University.', 'The',
'university', 'is', 'located', 'in', 'Rajkot,', 'Gujarat,', 'India.', 'I', 'enjoy', 'learning', 'about', 'machine',
'learning', 'and', 'deep', 'learning.', 'My', 'professor', 'teaches', 'AI', 'in', 'a', 'very', 'interactive',
'way.', 'We', 'perform', 'experiments', 'in', 'the', 'MA112', 'lab.', 'Python', 'is', 'my', 'favorite',
'programming', 'language.', 'I', 'have', 'completed', 'projects', 'on', 'virtual', 'mouse', 'and', 'smart',
'parking', 'system.', 'NLP', 'is', 'one', 'of', 'the', 'most', 'interesting', 'topics', 'for', 'me.', 'I', 'use',
'libraries', 'like', 'spaCy', 'and', 'NLTK', 'for', 'text', 'processing.', 'I', 'want', 'to', 'become', 'a',
'skilled', 'AI', 'engineer', 'in', 'the', 'future.']
```

Character at index 10:

d

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study Preprocessing of text (Tokenization, Filtration, Script Validation, Stop Word Removal, Stemming)	
Experiment No: 6	Date:	Enrolment No: 92410133039

2. **Write your observation for stemming and lemmatization you obtained for the sentences.**

stemming and lemmatization were applied to the given sentences to reduce words to their base forms. It was observed that stemming simply removes suffixes from words using basic rules, which sometimes results in incomplete or incorrect words. For example, words like “*studying*” were reduced to “*studi*” and “*learning*” to “*learn*”. While stemming is fast and simple, the output words are not always meaningful.

On the other hand, lemmatization produced more accurate and meaningful base forms by considering the context and grammatical structure of the sentence. For example, “*studying*” was converted to “*study*” and “*teaches*” to “*teach*”. Unlike stemming, lemmatization ensures that the resulting words are valid dictionary words.